Sisteme de programe pentru retele de calculatoare

1. Introducere

Un *sistem distribuit* este unul in care componente hardware si software localizate in calculatoare conectate in retea comunica si isi coordoneaza actiunile doar prin transfer de mesaje (Coulouris, Dollimore si Kindberg). Atunci cand reteaua care conecteaza calculatoarele intre ele este Internetul sau o parte semnificativa a acestuia, spunem ca sistemul *este distribuit pe scara larga*.

Sistemele distribuite constitutie unul din subiectele de mare atractie in stiinta calculatoarelor. Aceasta se datoreaza diversitatii aplicatiilor suportate, numarului mare de utilizatori si progreselor deosebite inregistrate in conceperea, proiectarea, implementarea, adminstrarea si utilizarea lor. Imbunatatirea sensibila a performantelor, in special a timpului de raspuns si a accesibilitatii, a stimulat cresterea majora a numarului de utilizatori apartinand unor categorii diverse (companii, institutii, guvern, utilizatori individuali si grupuri de utilizatori) ca si a numarului de aplicatii distribuite dezvoltate. La randul lor, utilizatorii au impus cerinte din ce in ce mai mari, inchizand astfel lantul de motive care stau la baza progresului continuu al domeniului.

Felul in care sistemele distribuite sunt construite depinde, evident, de cerintele aplicatiilor si implicit ale utilizatorilor, motiv pentru care prima parte a capitolului descrie cateva aplicatii semnificative si cerintele lor. In ciuda varietatii mari a aplicatiilor, se pot identifica cerinte comune uneia sau mai multor categorii, ceea ce face posibila simplificarea constructiei sistemelor distribuite prin dezvoltarea unor componente ce raspund simultan nevoilor mai multor aplicatii. Prezentarea problemelor comune generate de aceste cerinte si schitarea posibilelor solutii este facuta in a doua parte a capitolului.

1.1. Aplicatii distribuite

Dat fiind numarul mare si diversitatea de aplicatii distribuite pe scara larga, nu este posibila prezentarea lor, nici macar superficiala, in spatiul unui capitol introductiv. In sectiunea curenta vom aminti unele aplicatii distribuite (unele avand si importanta istorica) si vom analiza cateva dintre ele, pe care le consideram reprezentative pentru diferite categorii de aplicatii. Pe parcursul lucrarii, vor fi discutate si alte aspecte, legate de subiectele specifice abordate in capitolele sale.

WWW

Cea mai cunoscuta aplicatie distribuita pe scara larga este World Wide Web, WWW a carui functie primara este vizualizarea unor pagini continand in principal text si imagini, afisate de un program *client* (browser) pe ecranul calculatorului utilizatorului (http://en.wikipedia.org/wiki/Web_page). O pagina este regasita si transferata de la un *server*, program ce ruleaza pe un calculator diferit de cel al utilizatorului, dar aflat in conexiune cu acesta prin Internet. Fiecare resursa de la care se descarca pagina (mai multe explicatii despre resurse sunt prezentate in alte sectiuni ale cursului) are un identificator unic numit URL (Uniform Resource Locator), pe care clientul il foloseste pentru regasirea ei in Internet. Un exemplu este http://cs.pub.ro/pagini/ex.html, care specifica protocolul folosit de Web in comunicarea dintre client si server (http, HiperText Transfer Protocol), numele calculatorului pe care ruleaza serverul (*cs.pub.ro*) si identificatorul unic al resursei in calculatorul gazda (*/pagini/ex.html*). HTML (HiperText Markup Language) este limbajul in care se dau informatii (in forma unor marcaje) despre modul in care oricare browser trebuie sa afiseze continutul paginii. O pagina poate contine si hiper-legaturi (URL-uri) la alte pagini, situate in diverse servere in Internet. In Figura 1.1 sunt prezentate sursa HTML a unei pagini si felul in care este vizualizata de un browser. Marcajele, care sunt usor de recunoscut datorita includerii lor intre paranteze unghiulare, sunt intercalate cu texte.

```
<html>
<head> <title> AMALGAMATED WIDGET, INC. </title></head>
<body> <h1> Welcome to AWI's Home Page </h1>
<img SRC="http://www.widget.com/images/logo.gif" ALT="AWI Logo"> <br>
We are so happy that you have chosen to visit <b> Amalgamated Widget's</b>
```

```
home page. We hope <i> you </i> will find all the information you need here.
<p> Below we have links to information about our many fine products.
You can order electronically (by WWW), by telephone, or by FAX. </p>
<hr>
<h2> Product Information </h2>
<ul>
  <li> <a href="http://widget.com/products/big" > Big widgets </a>
  <li> <a href="http://widget.com/products/little" > Little widgets </a>
</ul>
<h2> Telephone Numbers </h2>
<ul>
<li> 1-800-WIDGETS
<li> 1-415-765-4321
</ul>
</body>
</html>
```

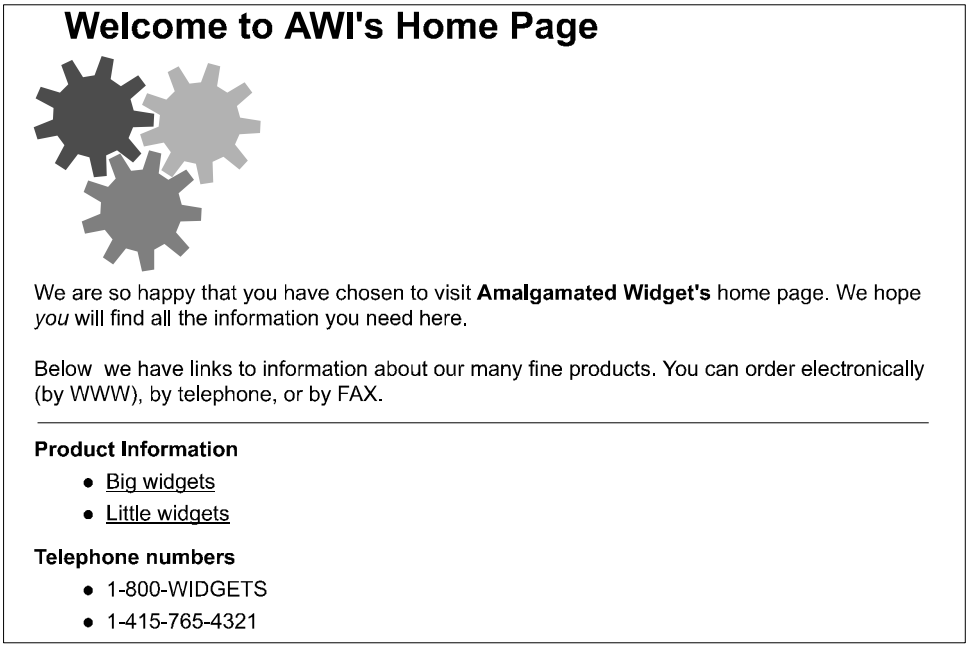


Figura 1.1

Interfata utilizatorului cu browser-ul este simpla si intuitiva, proiectata astfel incat utilizatorul sa o poata folosi fara o pregatire deosebita si sa poata naviga rapid de la o pagina la alta, oriunde in Internet. Astfel, in pagina curenta se pot scoate in evidenta (de exemplu printr-o culoare diferita de restul paginii) anumite texte sau imagini care au asociate legaturi la alte pagini. Activarea mouse-ului (click) pe una din zonele evidentiate are ca efect regasirea si afisarea paginii corespunzatoare. WWW are marele merit de a unifica interfata cu utilizatorul si de a o face independenta de tipul serverului. De asemenea, el nu este limitat la reprezentarea textelor si imaginilor, paginile putand include si alte tipuri multimedia precum audio si video sau codul executabil al unor programe.

URL-urile pot identifica nu doar resurse statice (al caror continut nu se modifica in timp) ci si programe care pot genera pagini HTML. De exemplu, la trimiterea de catre browser a unei cereri de pagina cu URL-ul http://cs.pub.ro/cgi-bin/myprog.cgi, serverul recunoaste ca *myprog* este un program si il pune in executie. Programul poate prelua datele de intrare din comanda transmisa de browser si poate genera o pagina HTML de raspuns, cu rezultatele executiei.

Unul din cele mai utilizate programe este motorul de cautare pe Web, care reprezinta un instrument indispensabil pentru ghidarea utilizatorilor in noianul sutelor de milioane de pagini disponibile pe Internet. Actualmente exista numeroase motoare de cautare, cele mai folosite fiind Google si Yahoo!. Pentru a gasi paginile ce contin informatia pe care o cauta, utilizatorul transmite motorului de cautare o intrebare (query) alcatuita din unul sau mai multe cuvinte grupate intr-o expresie sau combinate prin operatori logici ("sau", "si", "nu" …) si primeste ca raspuns o lista de URL-uri ordonate descrescator dupa gradul in care raspund intrebarii. Lista este dispusa in mai multe pagini, prima din ele fiind afisata direct utilizatorului, iar restul fiind usor accesibile prin mecanismul URL-urilor atasate, de regula, unor numere incluse pe fiecare dintre pagini. Utilizatorul se poate muta de la o pagina la alta prin simpla actionare a mouse-ului pe unul din aceste numere.

Conceptia WWW are la baza *accesul usor* al unui numar mare de utilizatori, la resursele Web raspandite pe scara larga in Internet. Aceasta cere ca reteaua sa ofere o *conectivitate* globala intre echipamentele de diverse categorii (inclusiv echipamente mobile) care gazduiesc browserele si orice calculator care gazduieste servere Web. Modul de acces nu trebuie sa depinda de distanta dintre client si server, cu alte cuvinte protocoalele folosite nu trebuie sa difere de la un caz la altul.

Conectivitatea este strans legata de *interoperabilitatea* intre clienti si servere. Cu alte cuvinte, trebuie asigurat nu doar transportul mesajelor in Internet, intre oricare client si oricate server Web, ci si intelegerea acestor mesaje indiferent de tehnologiile si implemetarile folosite la realizarea clientilor si serverelor, in conditiile in care Internetul include:

- hardware care foloseste reguli de reprezentare diferite ale tipurilor de date (de ex. big endian, little endian)
- sisteme de operare diferite implementeaza altfel protocoalele Internet; de exemplu, apelurile pentru comunicare de mesaje in Unix sunt diferite de cele din Windows
- limbajele de programare folosesc reprezentari diferite pentru caractrere si structuri de date (tablouri, inregistrari)
- implementari provenind de la dezvoltatori diferiti.

Solutia cea mai potrivita pentru asigurarea interoperabilitatii este adoptarea unor standarde (de exemplu pentru protocoalele de transport - TCP si pentru Web - HTTP) si respectarea lor la implementarea retelelor si Web-ului.

Sistemul trebuie sa ofere o *performanta* rezonabila, adica timpul de raspuns perceput de client nu trebuie sa varieze foarte mult de la o pagina la alta si sa se situeze in limite acceptabile pentru perceptia umana. Studii timpurii arata ca la un timp de raspuns de 0.1 secunde perceptia este ca sistemul reactioneaza imediat, iar o secunda este limita superioara la care utilizatorul ramane concentrat pe interactiunea cu sistemul, desi pierde senzatia de operare directa a datelor. Deoarece o parte importanta a timpului de raspuns este determinata de transportul mesajelor de cerere si de raspuns prin retea, satisfacerea acestor cerinte trebuie sa ia in consideratie distributia geografica a serverelor Web (de care depinde intarzierea de transfer a mesajelor) si performanta conexiunilor dintre servere si echipamentele de acces ale utilizatorilor interesati de diferitele informatii disponibile (de care depinde viteza de transfer). Replicarea resurselor pe mai multe servere Web este una din solutiile posibile pentru satisfacerea acestor cerinte. Ea are doua efecte importante. Mai intai, ea permite selectia unui server care este mai apropiat fizic de utilizator astfel incat intarzierea de propagare (pentru un cablu optic, aceasta este calculata ca raportul distanta/viteza luminii) sa fie cat mai mica si la care canalul de comunicatie are o capacitate mai mare astfel incat timpul de transmisie (adica dimensiunea mesajului/largimea de banda) sa fie mai mic. Apoi, distributia cererilor intre mai multe servere care pastreaza aceleasi resurse scade incarcarea pe fiecare server si reduce timpul de asteptare intre receptia unei cereri si inceperea tratarii sale (serverul memoreaza cererile in asteptare). Replicarea ridica insa si o problema dificila in cazul resurselor dinamice (care se modifica in timp), aceea a consistentei replicilor. Pe scurt, consistenta cere ca raspunsul la cererea unui client sa nu depinda de replica folosita. Desigur, sunt si alte metode de asigurare a scalabilitatii cum ar fi adoptarea unei comunicatii asincrone intre client si server, deoarece ea evita blocajele lungi caracteristice comunicatiei sincrone.

Cresterea performantei serverului (reducerea timpului de raspuns) poate fi realizata si prin executia paralela a mai multor taskuri, care trateaza astfel mai multe cereri in acelasi timp. Partea rea este ca taskurile

paralele pot concura pentru accesul la resurse comune, fiind necesara folosirea unor mecanisme de sincronizare care conduc la cresterea timpului de raspuns.

O problema legata de performanta este *scalabilitatea* sistemului, care desemneaza capacitatea acestuia de a functiona cu scaderi rezonabile de performanta in conditiile cresterii numarului resurselor Web si a numarului de utilizatori. Masurile de crestere a performantei mentionate anterior sunt utile si pentru asigurarea scalabilitatii sistemului.

O alta cerinta importanta este functionarea sistemului in conformitate cu specificatiile sale, chiar la producerea unor *defecte*, contribuind astfel la cresterea *disponibilitatii* sistemului. Defectele sunt destul de probabile in lumea imperfecta a retelelor, al caror regim normal de functionare este caracterizat de defectari partiale, ale unora din nodurilor care gazduiesc clienti si servere Web. La nivelul retelei care suporta traficul pe Internet, trebuie asigurate detectia pierderii sau modificarii datelor in timpul transportului si corectarea erorilor, precum si detectia defectarii nodurilor sau legaturilor si revenirea rapida la functionarea normala. Aceasta cerinta este tratata si la nivelul Web-ului, una din solutii fiind replicarea resurselor, pomenita mai inainte. De asemenea, solutia adoptata la nivelul protocolului HTTP nu obliga serverul sa coreleze diversele cereri succesive primite si simplifica atat functionarea acestuia cat si recuperarea mai simpla defectelor aparute la unele cereri, prin simpla retransmisie a acestora. HTTP are la baza un model "fara stare" (stateless), in care fiecare cerere contine toate informatiile necesare serverului pentru rezolvarea ei (este auto-continuta). O eroare la nivelul unei cereri nu se propaga, deci nu are efecte asupra celorlalte, pastrand astfel complexitatea problemei la un nivel redus.

Paginile Web sunt, de regula, accesibile neingradit tuturor utilizatorilor. Exista insa si pagini / grupuri de pagini protejate, la care accesul este permis doar anumitor utilizatori. Nu trebuie sa uitam ca WWW a aparut la CERN, fiind gandit ca instrument dedicat accesului cercetatorilor la articole si rapoarte de cercetare. Ne putem imagina usor ca el poate sluji si unor companii, care pun la dispozitia specialistilor angajati documente cu regim de acces privat. Aceste utilizari justifica cerinta ca anumite resurse Web sa fie protejate prin *controlul accesului*, deci pe baza unor credentiale de acces (de exemplu nume si parola). Aceasta cerinta poate fi completata de pastrarea *confidentialitatii* si a *integritatii* datelor transferate intre server si clienti, prin mijloace criptografice specifice.

Ca multe alte aplicatii distribuite, WWW este un sistem in continua evolutie nu doar ca numar de utilizatori, extensie geografica sau crestere a numarului de resurse. Se fac permanent *imbunatatiri ale performantei*, sunt modificate configuratiile etc. WWW este *deschis* extinderilor si re-implementarii in moduri diferite. Aceasta deschidere este asigurata prin publicarea interfetelor principale si conformitatea componentelor cu standardele publice. Ea este sustinuta, de asemenea, de o *arhitectura* potrivita, care structureaza WWW in componente cu functionalitati bine definite (cum sunt, de exemplu, clientul si serverul, dar si alte componente).

Aplicatii multimedia

O categorie de aplicatii foarte utilizata este cea care ofera descarcrea de continut multimedia (audio, video) pe Internet. De regula, aplicatiile sunt accesibile pornind de la un site Web (un exemplu este YouTube) care are un meniu mai bogat decat simpla vizualizare a unor filme sau ascultarea unor piese muzicale. Dar nu meniul ne intereseaza aici, ci cerintele specifice acestor aplicatii. De regula continutul multimedia este mult mai voluminos decat cel al unor pagini continand doar text si eventual imagini. Pentru un film, nu este practica descarcarea si memorarea lui in calculatorul utilizatorului si vizualizarea sa ulterioara. De regula, vizualizarea se face pe masura primirii continutului motiv pentru care receptia mesajelor trebuie facuta in ordinea in care acestea au fost transmise. Aceasta maniera de vizualizare are mai multe consecinte. Mai intai, ea cere *pastrarea vitezei de transfer* pe toata durata vizualizarii, astfel incat sa fie evitate discontinuitati cum ar fi "inghetarea" imaginii. Pentru acelasi motiv, eventualele erori aparute la transmiterea de la sursa la destinatar nu pot fi rezolvate prin re-transmiterea mesajelor, care ar strica ritmul de transfer si ar provoca discontinuitati in vizualizare. Rezolvarea acestor cerinte face apel la solutii diferte de cele folosite la descarcarea paginilor Web, cum ar fi: protocoale adaptate stilului particular de transfer multimedia, constituirea unor tampoane de memorare temporara a datelor sosite in flux si inceperea vizualizarii cu intarziere fata de primirea continutului (folosite pentru a compensa eventualele

discontinuitati in transmisie), generarea prin interpolare a unor cadre absente (folosita in locul re-transmiterii) si altele.

Cerinte diferite apar la transmiterea audio / video in timp real, in aplicatii precum Skype si videoconferinte. Ele se datoreaza deosebirii majore dintre descarcarea de continut multimedia, care se face intr-un singur flux (spre utilizator), aceste aplicatii in timp real se bazeaza pe fluxuri multimedia in ambele sensuri (spre si dinspre utilizator). Mai mult, aplicatiile audio/video in timp real folosesc si transmisii multicast (de la un utilizator catre toti ceilalti utilizatori cu care acesta este conectat la un moment dat). In plus, *constrangerile de timp sunt mai severe*, pentru a asigura interactiunea in timp real intre utilizatori. De exemplu, o intarziere de 100 msec este considerata potrivita pentru o interactiune rezonabila, in timp ce intarzieri de peste 300 msec fac aplicatiile neutilizabile (Stalings computer networks). Solutiile includ folosirea unor protocoale specifice transportului multimedia (cum este RTP, Real-Time Transport Protocol), rezervarea in retea a resurselor necesare pentru a obtine calitatea dorita a serviciului (RSVP, Resource Reservation Protocol), mecanisme de notificare a receptorului la deschiderea unei sesiuni multimedia si controlul sesiunii pe parcursul derularii aplicatiei.

Aplicatii Web compuse

In prezentarile anterioare, raspunsurile serverului sunt trimise direct utilizatorilor in forma paginilor Web. In multe cazuri, rezultatele cererilor utilizatorilor sunt insa transmise altor aplicatii Web. In primul caz formatul HTML este suficient pentru a descrie felul in care informatia trebuie afisata de browser, continutul ramanand a fi interpretat (inteles) de utilizatorul uman. In al doilea caz, aplicatiile Web trebuie sa inteleaga informatiile pe care le transmit una alteia, mai precis tipul si structura datelor transmise. HTML nu mai este suficient. In schimb, XML - eXtended Markup Language, care este un meta-limbaj de marcaje poate face acest lucru. XML poate fi folosit pentru definirea unor limbaje de marcaje pentru diferite tipuri de date. Curent, aceste limbaje bazate pe XML se numesc tot XML desi intre ele sunt diferente. Sintaxa XML seamana cu cea a HTML-ului, dar formatul lui poate fi atat inteles de oameni, cat si procesat de masina.

Reprezentarea datelor schimbate intre aplicatii nu este singura problema a acestor sisteme Web multi-aplicatie. In principiu, ele sunt o alternativa de dezvoltare a oricarei aplicatii Web ca o compunere a unor aplicatii mai simple si nu ca o entitate monolitica. Efectele unor aplicatii structurate in raport cu cele monolitice sunt importante si evidente, cateva dintre ele fiind: dezvoltarea mai rapida a aplicatiilor distribuite pe baza structurarii acestora si prin re-utiliarea unora din aplicatiile existente, suportul extensibilitatii prin posibilitatea de adaugare facila a unor noi aplicatii, intretinerea mai simpla si altele. Cum nimic nu este gratis pe lume, aceste avantaje sunt asociate si cu probleme. Stabilirea unor *modele arhitecturale* care sa permita structurarea ce raspunde cel mai bine criteriilor de proiectare a diverselor categorii de aplicatii este esentiala. Deoarece numarul aplicatiior Web dezvoltate este relativ mare si in continua crestere, definirea unor mecanisme care sa permita executia automata a unor actiuni precum descoperirea lor, aflarea modului in care pot fi invocate, compunerea automata a cererilor si receptia raspunsurilor devin o cerinta foarte importanta. In fine, cerintele mentionate anterior pentru Web si aplicatii Web raman valabile si pentru aplicatiile Web structurate.

Exista doua arhitecturi consacrate aplicatiilor Web, ambele bazate pe notiunea de *serviciu Web* folosita pentru a desemna o aplicatie individuala care ofera un serviciu specific accesibil de la distanta unei alte aplicatii Web. Prima, REST (REpresentational State Transfer) este arhitectura Web-ului. Ea are la baza doua tipuri de componente (clienti si servere) cu functionalitati distincte si un protocol de comunicare fara stare, HTTP. Cererea unui client contine toate informatiile necesare pentru ca serverul sa inteleaga ce trebuie sa faca (este auto-descriptiva). Aceasta constrangere creste vizibilitatea (un observator gaseste in cerere toate informatiile), fiabilitatea (recuperarea din defecte partiale se face mai simplu), scalabilitatea (serverul nu trebuie sa pastreze informatii de la o cerere la alta, dupa fiecare cerere elibereaza resursele folosite). In acelasi timp ea scade performanta (detaliile trebuie transmise cu fiecare cerere) si controlul serverului asupra aplicatiei (deoarece, acolo une este necesar, starea sesiunii este pastrata la client, iar controlul depinde de felul in care clientul gestioneaza starile). Cu toate astea, REST este utilizat pentru anumite categorii de aplicatii datorita unor facilitati importante:

- prevede mecanisme de crestere a eficientei cum ar fi replicarea datelor (cache-uri client si partajate) sau folosirea de servere proxy, care intermediaza accesul clientilor la resursele Web,

- defineste o interfata generala uniforma care permite identificarea resurselor si manipularea lor prin reprezentari XML,
- are un set redus de operatii (metodele GET, PUT, DELETE si POST din HTTP),
- suporta aplicatiile structurate pe mai multe nivele si
- poate extinde functionalitatea clientului prin descarcarea si executia de scripturi si appleti.

Setul redus de operatii ofera simplitate, dar manipuleaza intreaga stare a resursei, nu doar partea interesanta pentru client, care ar putea fi obtinuta prin invocarea unor operatii specifice.

Cea de a doua arhitectura este SOA, Service Oriented Architecture se bazeaza pe un limbaj de descriere a serviciilor Web, WSDL – Web Service Description Language.

e-Servicii

O categorie interesanta de aplicatii, aflata intr-o dezvoltare exploziva, este aceea a *serviciilor electronice* (e-services) livrate in special pe Internet si Web. In mod natural, conceptul de servicii electronice s-a dezvoltat in mediul privat, in forma afacerilor electronice si a comertului electronic, extinzandu-se apoi la mediul public ca servicii oferite de guvern si adminstratiile locale. Ca urmare, multe din serviciile electronice implica realizarea unor tranzactii prin Internet si comunicarea unor informatii importante pentru cumparatori sau cetateni. Serviciile electronice presupun o intelegere intre un furnizor si un client pentru furnizarea unui bun care nu este tangibil. Un exemplu tipic de astfel de serviciu este serviciul online de urmarire a pachetelor, oferit de FedEx clientilor ale caror pachete sunt transportate fizic tot de FedEx. Serviciul, care este complet automatizat, ofera clientilor informatii precise si de ultima ora despre pachetele in tranzit si notificari actualizate permanent despre data si ora livrarii. *Transparenta* realizata de FedEx prin transmiterea acestor informatii constituie unul din motivele succesului companiei in relatia cu clientii sai.

Servicile electronice s-au diversificat in timp, utilizarea lor in diverse domenii de activitate ducand la dezvoltarea mai multor categorii de e-servicii. Totodata, Web-ul a devenit mediul preferat de dezvoltare a serviciilor electronice. Aproape fara exceptie serviciile elactronice se bazeaza pe utilizarea altor servicii mai simple, unele din ale avand in spate instrumente informatice. O lista a celor mai reprezentative servicii si instrumente include servicii de monitorizare a activitatii utilizatorilor si a contextului (de multe ori bazate pe senzori), vizualizare a diferitelor obiecte multimedia, analiza imaginilor, baze de date, motoare de cautare, srvicii de comunicare in timp real inclusiv pentru continut multimedia, tele si video-conferinte, servicii de suport al activitatilor colaborative, servicii de control al accesului, servicii de securitate si servicii de administrare. Utilizarea acestor servicii poate fi optionala, recomandata sau critica. In continuare, descriem pe scurt doar cateva categorii de e-servicii impreuna cu cerintele specifice prin care se diferentiaza de celelalte categorii. La sfarsit prezentam cerintele comune tuturor e-serviciilor, completand astfel discutia facuta pe categorii de servicii.

e-Business si **e-commerce** sunt categorii inrudite. e-Business desemneaza servicii comerciale si de afaceri folosite de companii in eficientizarea legaturilor cu furnizorii si partenerii pentru a satisface mai bine necesitatile clientilor si a raspunde mai prompt asteptarilor lor. Ele cuprind intregul lant al activitatilor de afaceri, incluzand gestiunea lantului de aprovizionare, prelucrearea comenzilor, gestiunea serviciilor pentru clienti, cooperarea cu partnerii si altele. Inregistrarile tranzactiilor derulate stau la baza elaborarii unor rapoarte si analizei acestora cu programe specifice (analitics) in vederea imbunatatirii serviciilor. e-Commerce este considerat ca o parte a e-business-ului, care se refera la serviciile ce suporta tranzactiile de vanzare/cumparare a produselor comercializate si intarirea relatiilor cu clientii si furnizorii pentru a creste eficienta activitatilor comerciale. Tranzactiile sunt realizate cu alte companii (Business-to-Business, B2B), cu cumparatorii (Business-to-Consumer, B2C) sau cu guvernul (Business-to-Government, B2G).

Una din cerintele critice este *scalabilitatea* care, in cazul specific al serviciilor de afaceri si comert, se refera la capacitatea serviciului de a suporta traficul de varf, motivatia pentru aceasta interpretare particulara fiind evidenta. O alta se refera la *atomicitatea tranzactiilor*, care cere ca operatiile incluse intr-o tranzactie sa fie executate ca o singura operatie atomica. Daca executia atomica a unui grup de operatii este mai simpla la nivelul unui singur calculator prin folosirea unor instructiuni de sincronizare din repertoriul acestuia, ea este mai complicata la nivelul unor servicii distribuite.

Din punct de vederea al utilizatorilor, aplicatiile de e-business si e-commerce trebuie sa includa doua facilitati importante. Pentru clienti, *cautarea* in catalogul de produse trebuie sa fie simpla si sa tina seama de preferintele utilizatorilor. Pentru furnizor, *administrarea* si modificarea simpla a serviciului este la fel de importanta. Aceasta ar trebui sa permita, de exemplu, ca angajatii responsabili de promotii si reduceri de preturi sa poata face modificarile necesare fara a mai apela la personalul tehnic de administrare, scurtand astfel timpul si reducand costul unor astfel de operatii.

e-Banking se numara printre cele mai importante servicii oferite de banci clientilor individuali sau institutionali. Ele permit realizarea unor tranzactii financiare, vizualiarea si descarcarea unor informatii despre conturi (situatia financiara, extrase de cont, tranzactii recente, copii ale ordinelor de plata sau cecurilor etc.), transferul de bani intre conturi, plata unor facturi, angajarea unor plati periodice (pentru utilitati – apa, gaz, energie electrica), gestiunea unor conturi de economii, gestiunea informatiilor de profil, modificarea parolei si altele.

Una din cerintele critice este dezvoltarea unor servicii electronice *robuste* si cat mai *complete*. Efectul obtinut este creasterea increderii clientilor, anterior familiarizati cu contactul direct cu personalul bancilor, in serviciile bancare oferite online. Factorii care atrag clientii spre e-banking sunt in principal comoditatea, un control mai usor al conturilor, o organizare mai buna a datelor financiare si un cost mai mic datorat reducerii vizitelor la banca si eliminarii documentelor pe hartie. Legata de cerinta anterioara este extinderea ofertei de servicii, de exemplu dezvoltarea unor servicii personalizate si a unor servicii de notificare (a transferurilor bancare, a depozitelor, a scaderii sumei din cont sub un anumit prag), precum si livrarea serviciilor pe echpamente mobile. Extinderile si modificarile serviciilor ar trebui facute astfel incat sa conduca la reducerea costurilor bancii si la stabilirea unor preturi variabile pentru clienti, care sa fie adaptate numarului si complexitatii serviciilor oferite.

Cerintele de *fiabilitate* sunt deosebit de severe pentru e-banking. Datele si programeloe trebuie salvate periodic (back up) in memorii externe al caror nivel de fiabilitate este foarte ridicat, pentru recuperarea ulterioara din avarii. In cazul unor avarii de amploare, se folosesc resurse (situri) de rezerva, de regula situate in alte locatii decat sistemul folosit in operarea curenta, pentru reluarea rapida a serviciilor electronice. Pana la reluarea serviciilor electronice trebuie asigurata continuitatea activitatilor prin derularea serviciilor fara suport electronic. In ce priveste *securitatea* si *protectia*, accesul clientilor si al angajatilor trebuie controlat prin autentificare si autorizare (nume si parola). Este importanta autentificarea o singura data pentru toate serviciile bancare impreuna cu autentificarea serverului fata de client, limitarea duratei contactului clientilor cu aplicatiile bancare si folosirea criptarii in comunicarea dintre client si banca.

e-Government (cu derivatele sale mai recente e-democracy, e-participation, e-procurement) se refera la servicii oferite de autoritatile guvernamentale locale si statale pentru a *livra* cetatenilor si companiilor informatii despre activitatea lor, hotarari si regulamente, evenimente importante, locuri de munca libere, oportunitati de afaceri, a *purta un dialog* cu membrii guvernului / consiliului despre problemele cetatenilor si companiilor, solicitarile acestora si rezolvarea lor, a *consulta cetatenii* pentru luarea unor decizii importante, a *gestiona tranzactii* relative la acordarea de granturi, plata unor taxe, obtinerea unor autorizatii sau licente, licitatii publice si altele. Tendinta actuala este centrarea pe cetatean a acestor servicii, cu scopul de a-l angaja tot mai mult in guvernare si a-i oferi acele servicii si in acea maniera care sunt mai apropiate de interesele si profilul sau. Totodata, serviciile trebuie sa ia in consideratie varietatea utilizatorilor, care cuprinde, in afara localnicilor, rezidenti si imigranti interesati sa afle informatii despre noua lor patrie, angajati ai guvernului care folosesc serviciile in activitatea lor de fiecare zi, cetateni din alte tari care se documenteaza in vederea unei vizite etc.

Pentru a acoperi scenariile de utilizare curente, este necesara *integrarea serviciilor* in conformitate cu diverse modele de e-guvernare cunoscute: G2C (Government to Citizens), G2B (Government to Businesses), G2E (Government to Employees), G2G (Government to Governments), C2G (Citizens to Governments). De asemenea, *prezentarea multilingva* a serviciilor contribuie la satisfacerea cerintelor unui numar mai mare de utilizatori. In fine, *accesul usor* la diverse facilitati, cum ar fi formulare pentru diverse plati, documentatie, instructiuni de folosire, ar permite utilizarea mai larga a serviciilor de catre utilizatori care nu au o pregatire speciala.

e-Health grupeaza servicii de ingrijire medicala care includ asistenta medicala la distanta (tele-medicina), gestiunea fiselor medicale ale pacientilor (Electronic Health Record - EHR), gestiunea pacientilor (programarea consultatiilor, gestiunea grupurilor de medici etc.), tele-diagnoza (specialisti situati la distanta primesc date medicale despre un pacient si transmit asincron un diagnostic), tele-consultatii (medicii inspecteaza de la distanta, sincron, date despre pacient in forma digitala multimedia), tele-monitorizarea (supravegherea permanenta de la distanta a pacientilor pe baza datelor medicale colectate prin senzori si aparate specializate) si echipe virtuale de ingrijire medicala care partajeaza cunostintelor medicale despre pacienti, tratamente de succes, medicamente etc.), co-constructia si accesul la baze de cunostinte medicale si altele.

Constructia serviciilor are la baza instrumente de vizualizare si localizare, senzori medicali pentru monitorizarea pacientilor de la distanta, conferinte multimedia, servicii de analiza a imaginilor si servicii de suport al activitatilor colaborative. Se adauga la acestea portaluri cu informatii dedicate pacientilor si retele sociale pentru schimb de informatii intre pacienti privind stilul de viata, tratamente de succes si altele.

Cerintele sunt foarte diverse, chiar pentru un acelasi serviciu existand diferente foarte mari in functie de context. De exemplu, in situatii de urgenta sau in tele-chirurgie, cerintele privind *transmisia datelor* medicale ale pacientului (viteza, intarzierea, dispersia intarzierii, toleranta la pierderea pachetelor) sunt mult mai severe dacat in alte cazuri. De asemenea, *toleranta fata de erori* depinde si de formatul datelor transmise; de exemplu transmisia audio sau video este considerarata mai toleranta la erori decat transmisia EKG, care necesita masuri specifice de asigurare a integritatii datelor. Evident, identificarea corecta a contextului (care poate include o multime de factori cum ar fi temperatura, tensiunea, glicemia, pe langa cei consacrati, despre loc si timp) face apel la alte servicii specifice. O alta particularitate este utilizarea mai multor tipuri de canale de transmisie, de exemplu folosirea 3G (respectiv a canalelor mai evoluate precum EPS - Evolved Packet System) pentru supravegherea medicala a pacientilor la domiciliu sau pentru comunicarea intre sistemul informatic aflat pe ambulanta si spital sau centrul medical.

O atentie speciala este acordata *protectiei si confidentialitatii* datelor despre pacient, importanta subiectului fiind reflectata si de adoptarea unor directive europene relative la prelucrarea acestora in serviciile e-health. De exemplu, prelucrarea datelor medicale este permisa doar daca ea este ceruta de prevenirea, diagnosticarea, stabilirea tratamentului sau gestiunea serviciilor de ingrijire a sanatatii si daca aceasta este realizata de personalul medical cu respectarea secretului profesional. Prelucrarea se mai poate face daca pacientul si-a dat acordul explicit sau daca este necesara pentru protejarea intereselor vitale ale pacientului.

e-Learning cuprinde servicii de invatare si educatie online si reprezinta una din cele mai extinse si diversificate categorii, datorita numarului foarte mare de utilizatori individuali si institutii implicate, diversitatii tehnologiilor folosite (multimedia, forumuri, tele si video-conferinte, chat-uri, tabla inteligenta, ecrane partajate, clase virtuale si altele), modelelor folosite (auto-invatare, clase online, invatarea prin a face, invatare colaborativa etc.), instrumentelor de adminstrare (pentru dezvoltarea continutului, livrarea lui, gestiunea utilizatorilor, evaluarea performantei acestora etc.) si domeniilor de invatare. De altfel, fiecare din domeniile mentionate anterior necesita e-servicii specifice dedicate invatarii si cercetarii. De exemplu, derularea unor clase online in universitati presupune folosirea urmatoarelor servicii si instrumente: interactiuni sincrone (chat, whiteboard) pentru derularea lectiilor dinainte programate, precum si interactiuni asincrone (forum online, mail) pentru comunicarea in orice moment; suportul de invatare colaborativa (in grupuri de studenti) pentru rezolvarea in echipa a unor probleme sau elaborarea unor proiecte; materiale de invatare (carti, articole, rapoarte, note de curs) pentru pregatire individuala; instrumente de feedback si evaluare a nivelului de cunostinte al studentilor pentru urmarirea performantei lor si adaptarea sarcinilor pe urmatoarea perioada de invatare; instrumente de ghidare si gestiune a invatarii online; suportul strategiilor de invatare folosite de studenti pentru cresterea implicarii acestora in procesul de educatie.

m-Services sunt servicii din diversele categorii prezentate anterior care sunt livrate pe echipamente mobile. Caracteristica principala a m-serviciilor este mobilitatea, care atrage dupa sine o serie de facilitati cum ar fi: posibilitatea de acces la servicii spontan si in orice moment, deoarece echipamentul (de regula telefonul mobil) este in permanenta conectat la Web; utilizatorul poate accesa serviciile chiar in timpul unor

deplasari; utilizarea serviciilor este convenabila datorita numeroaselor facilitati disponibile actualmente pe mobile, cu care utilizatorul este familiarizat; eficienta utilizarii este sustinuta de progresele tehnologice in retelele fara fir (wireless). Problemele ridicate de serviciile mobile nu se rezuma la un alt fel de interfata cu utilizatorul, diferita de cea a echipamentelor ne-portabile. Una din cerintele importante este adaptarea la context a serviciilor oferite. De exemplu, un serviciu de alarmare ar trebui sa evite notificarea sonora daca utilizatorul se afla intr-o intalnire, evitand perturbarea evenimentului. Pentru a mentine o calitate buna a serviciilor, acestea trebuie adaptate la zestrea redusa de resurse a echipamentulelor mobile. Intre acestea, adaptarea prezentarii continutului la dimensiunile reduse ale ecranului (o solutie fiind limitarea spatiului dedicat reclamelor) poate contribui intr-o masura importanta la realizarea unei calitati care sa satisfaca asteptarilor utilizatorilor. In fine, mentinerea continuitatii serviciilor in prezenta unor deconectari temporare ale retelei este o cerinta specifica acestor servicii.

Cerinte comune serviciilor electronice

Datorita folosirii unor componente si tehnologii comune, cerintele Web-ului si aplicatiilor Web se regasesc la e-servicii. Rezumam aici lista cerintelor discutate anterior:

- utilizare simpla
- accesibilitate
- interoperabilitate
- performanta buna
- scalabilitate
- disponibilitate
- toleranta la defectari
- satisfacerea unor constrangeri de timp real
- controlul acesului
- confidentialitate si integritate
- deschidere la extinderi sau re-implementari

Aceste cerinte capata nuante usor diferite in cazul serviciilor. Acest lucru se datoreaza obiectivelor specifice e-serviciilor, dintre care doua joaca un rol primordial. Primul este *eficienta economica* pe care serviciile electronice trebuie sa o demonstreze in raport cu serviciile traditionale. Aceasta este usor vizibila in cazul e-commerce, unde clientul poate achizitiona produsele la preturi mai mici, iar furnizorul face o serie intreaga de economii pe lantul de comercializare. De exemplu, eliminarea magazinelor din acest lant poate aduce economii substantiale.

Oricum, eficienta economica este o cerinta care influenteaza proiectarea si implementarea serviciilor electronice. Rezolvarile includ adoptarea unor solutii tehnice cu costuri mici, dar si solutii care favorizeaza *cresterea numarului de clienti* prin servicii care raspund asteptarilor si determina cresterea satisfactiei acestora. Astfel, *adaptarea* dinamica a serviciilor la profilul clientilor sau, mai general, la contextul in care acestia se situeaza conduce la imbunatatirea serviciilor oferite. In plus, captarea si *analiza opiniilor* si *sentimentelor* acestora si modificarea conforma cu rezultatele acestor analize sunt solutii adoptate frecvent in proiectarea si intretinerea serviciilor. La fel de important, *designul adecvat* al site-urilor, o structura care faciliteaza navigarea usoara si regasirea informatiei, folosirea ingenioasa a graficii si a animatiei au ca rezultat usurinta de folosire de catre persoane fara pregatire speciala si de persoane cu dizabilitati. Alte masuri sunt livrarea serviciilor *oricand*, *oriunde*, si pe *orice echipament*, inclusiv pe mobile. In fine, un rol important il joaca *informarea* corespunzatoare a clientilor si *ghidarea* lor in folosirea serviciilor, compensand astfel contactul direct pe care clientii il au cu furnizorul de serviciile traditionale.

Pe langa pastrarea confidentialitatii si integritatii, prezente in toate aplicatiile distribuite, serviciile electronice au doua cerinte particulare deosebit de importante. Crearea si mentinerea *increderii* utilizatorilor sunt fundamentale pentru insasi existenta serviciilor si succesul lor in competitia cu serviciile traditionale, fata de care exista diferente notabile. Astfel, in cazul comertului traditional, clientul alege marfa inspectand produsele fizice si intra in posesia ei imediat ce face plata. In cazul comertului electronic, clientul ia cunostinta cu o marfa virtuala (imaginea produsului fizic) si, atunci cand face plata primeste doar promisiunea ca, in curand va primi si marfa dorita, de multe ori fiind obligat sa plateasca costul

transportului. Evident, un astfel de comert nu se poate desfasura decat pe baza increderii pe care clientii o acorda furnizorilor, care trebuie probata si intretinuta permanent de furnizor. De exemplu, furnizorii pot folosi servicii de urmarire a livrarii produselor care se informeze permanent clientii despre starea livrarii si pot publica opiniile clientilor despre produsele vandute sau despre serviciul prestat. Un avantaj al serviciile electronice este posibilitatea de colectare usoara si prelucrare statistica a informatiilor despre clienti si produse, rezultatele fiind folosite pentru cresterea calitatii acestor servicii.

A doua cerinta este pastrarea *intimitatii*, adica pastrarea confidentialitatii informatiilor personale si de profil ale clientilor, pe care siturile de servicii electronice le colecteaza prin formulare completate chiar de clienti sau prin urmarirea interactiunilor online dintre clienti si servicii (pagini vizitate, durata interactiunilor, cuvinte cheie folosite la cautari pe site si altele). In cazul comertului electronic, informatiile despre cumparaturile facute de un client pot fi folosite pentru a adapta reclamele si promotiile la preferintele clientului sau pentru ajustarea costului in raport cu sumele anterioare cheltuite de client pentru bunurile achizitionate. Multe informatii sunt colectate fara stiinta clientului. Exista si reglementari asupra categoriilor de informatii protejate, dar si diferente intre felul in care diferite regulamente stabilesc modul in care datele pot fi folosite. Totodata, reglementarile privind protectia nu se aplica daca informatiile au fost colectate cu acordul clientului, ceea ce complica destul de mult lucrurile. Desi putine situri de servicii electronice iau in consideratie problemele intimitatii, exista o recunoastere generala ca acestea sunt importante, cautandu-se solutii adecvate pentru rezolvarea lor.

Un serviciu electronic nu este o constructie monolitica. De regula, el este un ansamblu structurat de servicii software. O astfel de arhitectura raspunde catorva cerinte importante. Usurinta de proiectare, implementare si intretinere provine din structurarea serviciilor in componente mai simple intre care sunt definite relatii clare. Este astfel posibila inlocuirea unei componente cu o alta mai performanta, care are aceeasi functionalitate si expune aceeasi interfata celorlalte componente ale sistemului, ca si *extinderea* sistemului cu alte servicii si facilitati. Organizarea permite partajarea acelorasi componente de catre servicii diferite, contribuind la eliminarea redundantelor si la o prezentare uniforma si coerenta catre utilizator a ansamblului de servicii.

Un ultim subiect la care ne referim aici este legat de *disponibilitatea* serviciilor. O disponibilitate buna inseamna ca, cu o probabilitate foarte mare, serviciul functioneaza conform specificatiilor atunci cand este invocat. Defectarea temporara a unor parti ale sistemelor (hardware sau software) afecteaza desigur disponibilitatea. Mai sunt insa si situatiile, mai frecvente decat defectarile, in care se aduc schimbari serviciilor. Acestea pot viza atat partea de programe cat si partea de date. O cerinta importanta este ca serviciile sa ramana disponibile atat in cazul defectelor cat si in perioadele in care se fac modificari, tehnica replicarii putand fi folosita cu succes in ambele cazuri.

- Sisteme bazate pe Web
  - Problema: aplicatii complexe oferite printr-o interfata uniforma, usor de folosit si intretinut
  - Solutia:
    - browser Web ca interfata
    - aplicatii distribuite, in spatele serverului Web
    - Ulterior: aplicatie (puternic interactiva) impartita intre server si client (RIA – Rich Internet Applications)

Cloud computing

Daca e-serviciile se adreseaza unor domenii diverse, diferite de Tehnologia Informatiei (TI), *Cloud computing* vine cu o idee noua, propunand servicii de infrastructura, platforma si software care se adreseaza chiar domeniului TI. In locul cumpararii unor produse casnice, de imbracaminte, carti etc. ce fac obiectul e-comertului, Cloud computing ofera inchirierea unor calculatoare si retele, baze de date si sisteme de operare, sau software de aplicatii. Aceste servicii sunt accesibile pe Internet, sunt utilizate "*la cerere*", iar plata se face in functie de consum, pe baza unui *contract* intre furnizori si consumatori. Atractia oferita de Cloud computing pentru clienti este inlocuirea achizitiei si operarii unor resurse scumpe cu inchirierea si utilizarea lor la preturi considerabil mai mici. Operarea, intretinerea, actualizarea versiunilor si altele cad in sarcina furnizorului.

Multe din cerintele Cloud computing sunt similare celor ale serviciilor. Exista insa si cerinte noi generate de modelul de afaceri adoptat, care este bazat pe o *intelegere* (SLA, Service Level Agreement) intre client si furnizor privind volumul resurselor (putere de calcul, capacitatea memoriei externe) si calitatea serviciilor (timp de raspuns, numar de cereri rezolvate pe secunda). Furnizorii fac eforturi pentru a scade costul serviciilor si a ramane competitivi pe piata, prin exploatarea unor metode care asigura *elasticitatea* sistemului. Aceasta presupune nu doar cresterea teoretic nelimitata, la cererea clientului, a volumului resurselor alocate (scalarea "in sus"), dar si scaderea volumului resurselor daca, pe parcursul serviciului, cererea de resurse devina mai mica (scalarea "in jos"). Economia se poate obtine si cu alte metode, cum ar fi o mai buna *planificare* a resurselor, reducerea consumului de *energie* prin oprirea temporara sistemelor nefolosite etc.

Atractivitatea Cloud computing poate fi crescuta de furnizori prin adoptarea unor *modele de plata* diverse, cum ar fi rezervarea prealabila a resurselor, licitarea resurselor incomplet folosite de alti utilizatori, ghidarea utilizatorilor in dimensionarea serviciilor pe care le solicita si altele.

Una din problemele delicate este *securitatea* datelor memorate in depozitele de stocare ale Cloud-ului. De exemplu, nu este evident ca operatiile de stergere din depozite a informatiilor clientilor au efect real si nu permit ca datele respective sa fie pastrate si folosite ilegal de furnizor.

Aplicatii stiintifice

O categorie aparte este cea a aplicatiilor de calcul de inalta performanta (High Performance Computing), caracterizate de calcule intensive aplicate de multe ori unor volume mari de date. Ele sunt cu preponderenta aplicatii stiintifice si ingineresti, dar nu ocolesc nici nevoile intreprinderilor (companiilor) de proiectare a unor produse complexe (de exemplu, in industria automobilelor), de extragere a unor cunostinte noi din volume mari de date (data mining), de analiza a unor noi produse sau procese prin simulare, de analiza a riscului in vederea luarii unor decizii asupra afacerior si altele.

Deoarece performanta inalta, care este obiectivul principal al acestor aplicatii se atinge prin *paralelizarea* codului, o directie demarata in urma cu mai multi ani este conceperea si realizarea unor sisteme de calcul paralele, care ating performante cu cateva ordine de marime mai mari decat sistemele secventiale de putere comparabila. Pentru aceasta, programele sunt structurate in procese (task-uri) sau fire de executie (thread-uri) care ruleaza in paralel si comunica intre ele prin date aflate intr-o memorie comuna. Desigur, procesele si firele de executie trebuie sa se sincronizeze intre ele pentru a evita producerea unor rezultate gresite. Exista o mare varietate de arhitecturi paralele de calcul.

O solutie din ce in ce mai folosita este distributia datelor pe mai multe calculatoare si prelucrarea lor in paralel de catre procese/procesoare diferite. Comunicarea intre procese executate de calculatoare diferite se face prin mesaje transmise prin reteaua de comunicare ce leaga intre ele calculatoarele. Retelele pot diferi de la o arhitectura la alta. De exemplu, clusterele (colectii de calculatoare de obicei conectate prin retele locale) pot folosi o retea bazata pe TCP pentru traficul de management (de exemplu transmiterea unor informatii despre retea si calculatoarele din cluster) si una de viteza ridicata (care poate fi bazata pe TCP sau pe alte protocoale, ca in cazul InfiniBand sau Myrinet) pentru traficul computational.

Pe langa cerinta de performanta, prelucrarea paralela pe sisteme distribuite reclama instrumente adecvate pentru dezvoltarea aplicatiilor: limbaje de programare, medii de dezvoltare etc. O solutie foarte raspandita este MPI (Message Passing Interface), un standard de facto pe baza caruia au fost dezvoltate biblioteci software dedicate programarii paralele. O realizare mai noua este definirea modelului *MapReduce.*

Cresterea performantei retelelor a facut posibila extinderea distantei intre calculatoare si, prin generalizare, intre clustere, peste limitele retelelor locale, situatie pe care o intalnim la *Grid*. Ideea de baza a fost folosirea resurselor incomplet utilizate, pentru a rula aplicatii stiintifice in perioadele de ne-utilizare de catre proprietari. Mai mult, Grid-ul a fost lansat impreuna cu conceptul de *Organizatie Virtuala*. Aceasta cuprinde un grup de utilizatori apartinand unor organizatii fizice, care partajeaza resursele puse la dispozitie de acestea pentru a lucra impreuna la un proiect. Desi poate avea structura unei organizatii

fizice, Organizatia Virtuala nu are un sediu fizic, iar colaborarea intre membrii sai se deruleaza prin Internet. In schimb, ea are o componenta dinamica (membrii ei se pot retrage sau pot reveni oricand in organizatie). In plus, ansamblul de resurse disponibile este variabil in timp, iar folosirea resurselor se supune constrangerilor impuse de proprietari (cine si cand poate utiliza resusele). Dubla dinamicitate a Grid-ului face ca *folosirea eficienta a resurselor* si *cresterea calitatii aplicatiilor* sa fie deosebit de dificile.

Retele peer-to-peer (P2P)

Aplicatiile prezentate anterior se bazeaza pe specializarea componentelor in doua clase distincte, cu functionalitati bine precizate, cea a *clientilor* si cea a *serverelor*. Desi are contributii incontestabile in dezvoltarea multor aplicatii distribuite, modelul nu este intotdeauna convenabil. Sa ne gandim la o aplicatie foarte cunoscuta azi, de distributie a unor fisiere multimedia, in care fiecare utilizator ofera celorlalti astfel de fisiere si, desigur, primeste de la ceilalti fisiere multimedia. Caracterul benevol al participarii la astfel de schimburi si dorinta de evitare a unui "Big Brother" care sa controleze schimburile de fisiere fac nepotrivita solutia client-server. In schimb, modelul *peer-to-peer* adoptat corespunde mai bine dezideratelor mentionate. Ideea este ca, daca un utilizator doreste un fisier, el il poate primi direct de la utilizatorul care il ofera. Modelul nu mai face diferenta functionala intre clienti si servere, fiecare nod al retelei (sistemului) fiind in acelasi timp client si server. Retelele P2P sunt folosite in calcule intensive (de exemlu SETI@home - Search for Extraterrestrial Intelligence), colaborare in timp real intre utilizatori (de exemplu chat - Internet Relay Chat), distributie de continut (Napster, Gnutella, Freenet, BitTorrent) si altele.

Realizarea retelelor P2P reclama gasirea solutiilor la intrebari de genul: cum se formeaza astfel de retea, cum pot nodurile sa intre sau sa iasa din retea, cum se mentin legaturile intre noduri, cum se face regasirea nodului care contine un anumit fisier si altele. Desigur, nu sunt absente cerintele relative la o functionare eficienta, la tolerarea erorilor, la incredere si securitate.

1.2. Avantajele sistemelor distribuite

Sistemele distribuite prezinta avantaje incontestabile. Astfel reducerea *timpului de executie* a aplicatiilor prin paralelizare si distributie reprezinta un avantaj important pentru categorii diverse de aplicatii. Utilizatorii aplicatiile bazate pe calcule intensive asociate eventual cu volume mari de date obtin rezultatele asteptate intr-un timp rezonabil, de ordinul orelor. In cazul aplicatiilor tranzactionale si colaborative, cresterea performantei este o conditie pentru pastrarea interactiunilor la un nivel corespunzator perceptiei umane.

Un alt avantaj este cresterea *tolerantei la defectari*. Deoarece in cazul sistemelor distribuite aceste defectari sunt partiale, solutia folosita este replicarea datelor sau a calculelor. Daca una din replici se defecteaza, foarte probabil o alta poate fi folosita in locul ei.

Unele aplicaţii pot fi împărţite în faze cu funcţionalităţi distincte: de calcul în virgulă mobilă, de prelucrări grafice, de prelucrare de fişiere, de imprimare etc. *Specializarea functionala* permite exploatarea mai bună a caracteristicilor diferitelor componente ale sistemelor distribuite pentru realizarea unor performanţe deosebite (folosirea unor procesoare în virgulă mobilă, a unor procesoare grafice, a unor servere de fişiere sau de imprimare).

Un avantaj exclusiv al sistemelor distribuite este usurinta de *acces la resurse indepartate*. Ea este utila in aplicatii inerent distribuite, asa cum sunt multe din aplicatiile discutate anterior. Accesul usor la distanta se datoreaza naturii inerent distribuite a sistemelor si avansurilor tehnologice in domeniul retelelor, care ofera de multe ori o performanta rezonabila chiar in asemenea situatii. Acolo unde este nevoie de o performanta mai buna, poate veni in ajutor replicarea resurselor si plasarea replicilor astfel incat sa favorizeze accesul unei parti insemnate a utilizatorilor.

Sistemele distribuite permit *cresterea incrementala*. Puterea de calcul poate fi crescuta pe masura nevoilor utilizatorilor, iar componentele depreciate pot fi inlocuite mai usor. Cresterea incrementala permite si o planificare mai judicioasa a investitiilor in tehnica de calcul.

Sistemele distribuite permit nu doar *partajarea* datelor, ci si a *resurselor* de calcul hardware si software, ceea ce creaza premisele pentru o utilizare mai eficenta a acestora. Alaturi de scaderea in timp a pretului componentelor, partajarea resurselor conduce la un *raport cost / performanta* mai redus decat sistemele mari, centralizate.

1.3. O arhitectura a sistemelor distribuite

Aplicatiile distribuite sunt constituite din mai multe componente software situate in sisteme diferite, care inter-opereaza prin interemediul retelei. Mai precis, un program situat intr-unul din sisteme are acces la programe si date din alte sisteme. Din pacate, platformele (hardware si sistem de operare ale) sistemelor individuale utilizate in constructia sistemelor distribuite au la baza un model conceptual si tehnologii concrete care nu au avut in vedere calculul distribuit, astfel ca ele nu ofera solutii convenabile pentru inter-operare. In plus, platformele sunt, de regula, eterogene astfel ca interfetele pe care le ofera programatorilor de aplicatii (API-uri) pot diferi de la un producator la altul.

O varianta foarte populara de rezolvare este includerea, intre nivelul aplicatiilor si cel al platformei, a unui nivel suplimentar numit *middleware* (denumirea este motivata de pozitia de mijloc ocupata), care ofera serviciile cerute de aplicatiile distribuite (pe care infrastructura nu le ofera), prin interfete (API-uri) standard.

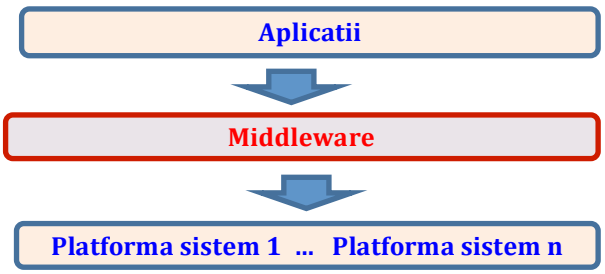


Figura 1.2

Deci, rolul pe care middleware il are este sa acopere golul dintre oferta platformei si cerintele aplicatiilor distribuite. Serviciile middleware sunt desigur legate de functionalitatile si trasaturile sistemelor distribuite si vizeaza dezvoltarea mai simpla a aplicatiilor distribuite. Mai precis, ele permit dezvoltatorului de aplicatii sa se concentreze pe proiectarea si implementarea logicii acestora si nu pe rezolvarea problemelor de comunicare la distanta intre componente, acces concurent la resurse, tolerarea defectarilor, controlul accesului si altele. Middleware scuteste programatorii aplicatiilor de problemele mai neplacute ale programarii de nivel scazut (de exemplu interfata sistemului de operare), iterfetele (API-urile) de nivel mai ridicat oferite fiind mai apropiate de necesitatile acestora.

Middleware realizeaza coordonarea si *interoperarea* diferitelor componente ale sistemelor distribuite, asigurand interoperabilitatea componentelor care provin de la diversi furnizori de tehnologie. Un acelasi serviciu poate avea implementari diferite, dar toate trebuie sa respecte descrierea interfetei care defineste serviciul. Aceasta permite ca implementarea unui serviciu sa fie inlocuita cu o alta mai performanta, schimbarea fiind transparenta pentru aplicatiile care utilizeaza serviciul. Serviciile sunt *portabile*, deci pot fi transferate de la o platforma la alta cu un efort redus. Oricum, interfata pe care un serviciu o ofera aplicatiilor este independenta de platfoma pe care acesta ruleaza.

Middleware ofera mai multe categorii de servicii. Desi nu exista o clasificare oficiala, sunt general recunoscute categorile la care ne referim in continuare.

Comunicare. Unele servicii imbogatesc formele de *comunicare*, oferind facilitati suplimentare socketilor, care transmit mesaje avand forma unor simple secvente de octeti. De exemplu, comunicarea prin apeluri de proceduri intre programe care ruleaza eventual in calculatoare diferite, este realizata de RPC (Remote Procedure Call), care face ca programarea aplicatiilor distribuite sa semene foarte mult cu ca a aplicatiilor

ce ruleaza pe un singur calculator. Acest stil de programare este promovat de Java RMI (Remote Method Invocation), prin invocarea de metode intre obiecte din aceeasi masina virtuala Java (JVMs) sau din masini diferite situate in calculatoare distincte. Comunicarea unor date cu structura complexa este facilitata de tehnicile de serializare a parametrilor si chiar a unor obiecte intregi (marshal si unmarshal). Pentru aplicatii Web, comunicarea de date cu tip si structurate intre aplicatii scrise in diferite limbaje de programare se bazeaza pe XML (eXtensible Markup Language), care permite reprezentarea datelor ca texte, intercalate cu marcaje care dau informati despre tipul acestora. Alte servicii de comunicare se refera la mesageria peer-to-peer, cozile de mesaje, posta electronica, interschimbarea electronica a datelor si altele.

Reprezentarea datelor. Intr-un sistem distribuit, programele aflate in calculatoare diferite schimba intre ele date cu tip. Reprezentarea datelor poate diferi de la un program la altul. Astfel, nu toate calculatoarele folosesc standardul IEEE 754 pentru numerele in virgula mobila, intregii sunt reprezentati pe cuvinte de dimensiuni diferite (16, 32 sau 64 de biti), in forma big-endian (cel mai semnificativ bit este in octetul cu adresa mai mare) sau cea little-endian (cel mai semnificativ bit este in octetul cu adresa mai mica) care difera de precedenta. Diferente de reprezentare pot fi introduse nu doar de hardware. Compilatoare diferite pot produce reprezentari diferite ale datelor cu tip, chiar daca aplicatiile folosesc acelasi limbaj de programare.
Pe langa aceste aceste diferente, mentionam ca reprezentarile folosite de programe nu sunt potrivite pentru transmiterea datelor prin retea. In solutiile adoptate curent pentru problemele generate de diversitatea formatelor (de exemplu, RPC si Java RMI), se utilizeaza un format de date distinct pentru transfer. Acesta este folosit de programul transmitator pentru a codifica datele din formatul propriu (numit si format de prezentare) intr-un mesaj mesaj reprezentat in formatul de transfer, pe care il transmite prin retea. La randul sau, programul receptor va decodifica mesajul primit, obtinand datele in formatul de prezentare propriu.

Sincronizarea intre procese. Executia unei aplicatii distribuite se materializeaza in forma unei colectii de procese (si fire de executie – thread-uri) care ruleaza in paralel si pot concura pentru utilizarea unor resurse comune. De asemenea, procese din aplicatii diferite pot *concura* pentru aceleasi resurse. Concurenta nu trebuie sa produca comportamente haotice si nu trebuie sa conduca la stari inconsistente ale resurselor comune. Middleware face controlul concurentei, de regula prin *excludere mutuala* distribuita, prin care se asigura ca, la un moment dat, o resursa este folosita de cel mult un proces. Sunt si situatii in care procesele pot accesa simultan resursele. Un exemplu clasic este folosirea partajata a unei memorii pentru citirea concurenta a acelorasi locatii, in timp ce folosirea exclusiva este dedicata scrierilor. In alte cazuri, procesele pot *colabora* pentru realizarea unui obiectiv comun. O situatie frecventa este cea a *tranzactiilor*, care pot pune in joc procese din calculatoare diferite. Pentru executia corecta a unei tranzactii, trebuie ca fiecare din procesele participante sa execute corect partea sa de operatii. Pentru a evita erorile provocate de eventualele defectari in timpul executiei, tranzactia se executa in regim atomic. Aceasta inseamna ca fie toate procesele executa operatiile lor, fie nici un proces nu executa vreo operatie. Middleware ofera si *servicii de timp*, precum si *ordonarea executiei* unor actiuni in functie de momentul la care s-au produs anumite evenimente care le declanseaza.

Servicii de directoare. Accesul la entitati (date si programe) din diferite sisteme presupune cunoasterea adreselor acestora in retea, respectiv a adreselor de retea si numerelor de porturi asociate punctelor de acces. In programarea actiunilor cu aceste entitati, se folosesc *numele* entitatilor (succesiuni de caractere), care sunt mai usor de gestionat si de tinut minte de programatori. Daca numele atasat unei entitati este unic, el poate fi folosit la *identificatea* entitatii: numele (identificatorul) o diferentiaza de oricare alta entitate din retea, deci acelasi nume nu poate fi asociat cu doua sau mai multe entitati diferite. Trecerea de la nume la adresa entitatii (de fapt la adresa punctului de acces la entitate) se face la executia programelor, de catre un *sistem de nume*. Folosirea numelor permite ca adresele entitatilor sa se modifice (de exemplu, in cazul obiectelor mobile) fara ca ceasta sa afecteze accesul ei de alte entitati. Cu alte cuvinte, numele este independent de locatia entitatii. In multe situatii, aplicatiile sunt interesate de un serviciu care rezolva o anumita problema, nu de o entitate anume care face acest lucru. Pentru astfel de situatii, middleware ofera servicii de *directoare*, cum ar fi cel de *publicare/ subscriere/notificare* (PSN) a serviciilor. PSN pastreaza specificatii ale serviciilor disponibile (sau publice). O aplicatie informeaza serviciul PSN ca este interesata de un serviciu cu o anumita specificatie (spunem ca *subscrie* la un astfel de serviciu). Cand un astfel de serviciu devine disponibil (este publicat), PSN *notifica* aplicatiile interesate, acestea putandu-le folosi in continuare. Un alt serviciu este cel de *cautare*, care pastreaza informatii despre serviciile disponibile.

Atunci cand o aplicatie are nevoie de un serviciu, invoca serviciul de cautare si afla identitatile serviciilor care corespund cererintelor sale. Folosind criterii suplimentare (cum ar fi cele de performanta), aplicatia va selecta unul din aceste servicii.

Gestiunea datelor. O alta problema importanta este organizarea datelor distribuite. Deoarece organizarea tipica, in fisiere gestionate de sistemul de operare nu este convenabila multor aplicatii, au fost dezvoltate alte forme de organizare, care sa faciliteze accesul la date si sa permita cautari si interogari eficiente ale depozitelor distribuite de date. Pe langa clasicul model *relational*, au fost adoptate modele care sa corespunda mai bine cerintelor aplicatiilor si sa simplifice tratarea interogarilor. Un exemplu este modelul *cheie-valoare*, utilizat in multe aplicatii actuale in care cautarea simpla a datelor, dupa o singura cheie este suficienta. Middleware ofera suportul pentru stocarea, cautarea si gestiunea depozitelor distribuite de date, organizate conform acestor modele.

Tratarea defectarilor. Middleware ofera mai multe solutii generale pentru tratarea defectarilor sistemelor distribuite. Unele se bazeaza pe faptul ca defectele sunt partiale, fiind posibila folosirea replicarii datelor sau proceselor pentru tolerarea acestora. Oricum, deoarece replicile sunt inevitabil actualizate la momente diferite de timp, solutile de replicare trebuie asociate cu cele de asigurare a consistentei replicilor. Alte solutii se bazeaza pe salvarea unor *amprente* succesive ale starii sistemului si folosirea acestora pentru *recuperarea* din erori cu costuri suplimentare reduse. Aceste solutii implica existenta unor servicii middleware pentru detectia erorilor si pentru coordonarea revenirii la functionarea normala. O abordare diferita este inzestrarea unor servicii larg utilizate cu capabilitati de toleranta la defectari, cum ar fi cele pentru comunicarea multicast sau pentru tranzactii. Solutiile obtinute in acest mod sunt mai simple, mai bine adaptate serviciilor pe care le imbogatesc si mai eficiente.

Securitatea. Asigurarea securitatii unui sistem distribuit are un specific aparte, datorita faptului ca ea se refera la un sistem ale carui componente (sisteme de calcul si retele) sunt deja inzestrate cu propriile lor solutii de securitate. Serviciile trebuie sa raspunda unor cerinte specifice ansamblului acestor componente si la aplicatiile distribuite suportate. De exemplu, identificarea utilizatorilor trebuie sa fie valida la nivelul intregului sistem iar autentificarea trebuie facuta o singura data, desi aplicatia se bazeaza pe folosirea mai multor sisteme cu politici de administrare diferite. Similar, detectia intruziunilor isi bazeaza solutiile pe corelarea informatiilor culese in diverse componente ale sistemului distribuit, care au elemente comune cum ar fi folosirea lor in aceeasi aplicatie distribuita. Serviciile curente se refera la controlul accesului (autorizare), auditarea pentru detectia intruziunilor si anomaliilor, criptarea, semnaturi digitale si altele.

Gestiunea sistemului distribuit. Alte servicii se refera la gestiunea diferitelor resurse ale sistemului distribuit (procesoare, memorie, depozite de date) avand in vedere criterii globale cum ar fi *eficienta* utilizarii resurselor sau *optimizarea performantei* oferite anumitor aplicatii. Solutiile se refera la rezervarea, alocarea si planificarea resurselor la nivelul sistemului distribuit, monitorizarea functionarii sistemului, adaptarea algoritmilor de planificare in functie de datele de monitorizare si altele. Pentru sistemele bazate pe *evenimente*, middleware ofera servicii de colectare a informatiilor despre diversele evenimente generate de senzori, utilizatori si aplicatii, de procesare a lor (filtrare, generarea unor evenimente complexe) si de livrare catre acele componente si aplicatii ale caror actiuni sunt declansate de producerea lor. Similar, pentru sistemele dependente de *context*, middleware include servicii pentru determinarea contextului si notificarea componentelor si aplicatiilor ale caror actijuni sunt dependente de acest context.

Cadre de lucru (Frameworks). Designul adecvat al componentelor (de exemplu, proiectarea unor componente generale, flexibile, folosind tipare de succes ce solutioneaza clase de probleme si nu probleme particulare) deschide drumul spre reutilizarea acestora in diverse aplicatii, efortul mutandu-se astfel de la dezvoltarea unor componente noi, la integrarea unora existente. Desigur, acestea contribuie la cresterea calitatii aplicatiilor, iar daca luam in consideratie si eventualitatea folosirii unor *cadre de lucru* (frameworks) se obtine si reducerea efortului de dezvoltare a acestora.

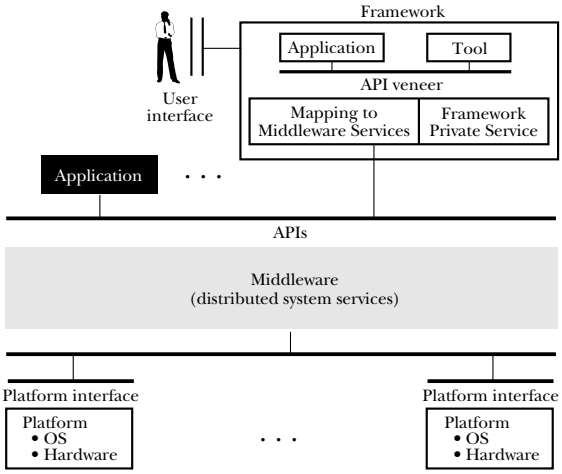


Figura 1.3

Un cadru de lucru este un mediu software care usureaza dezvoltarea aplicatiilor dintr-un anumit domeniu. El este definit printr-un API, o interfata pentru utilizator si un set de instrumente. Un cadru de lucru foloseste servicii middleware externe, selectand din serviciile disponibile pe acelea care corespund profilului aplicatiilor. In plus, el poate simplifica interfata acestor servicii (API mai simplu) renuntand la facilitatile care nu se potrivesc acestui profil. In plus, cadrul poate include si propriile sale servicii. In fine, instrumentele faciliteaza folosirea cadrului de catre utilizatori, programatori si manageri de sistem.

Referinte

1. A.S. Tanenbaum, M. van Steen
Distributed Systems. Principles and paradigms, Ed.2
Prentice Hall 2007
2. George Coulouris, Jean Dollimore,Tim Kindberg, Gordon Blair
Distributed Systems. Concepts and Design (Fifth Edition)
Addison-Wesley, 2012
3. L. Shklar, R. Rosen
Web Application Architecture
John Wiley & Sons, 2003
4. E. Cerami
Web Services
O'Relly 2002
5. Larry L. Peterson and Bruce S. Davie
Computer Networks a system approach
Elsevier 2012